

Multicore and Distributed Systems

Lecture 15

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Jorge Coelho (jmn@isep.ipp.pt)

Recap from Lecture 14

- Manual vs. automatic memory management
- Reference counting vs. Tracing
- Mark-and-Sweep, Mark-Compact, Semi-space, Mark-Region

Recap from Lecture 14

Feature	Mark-and-Sweep	Mark-Compact	Semi-space	Mark-Region
Basic Principle	Mark live objects, sweep dead ones into free list	Mark, then compact live objects	Copy objects between two memory spaces	Mark and free entire unused regions
Memory Fragmentation	High	Low	None	Low
Memory Overhead	Low	Low	High (double space)	Medium
Allocation Speed	Slower (free-list)	Fast (Bump)	Fast (Bump)	Fast (Bump)

GC Trade-offs

- **Throughput: % of total execution spent running the app (higher is better).**
- **Latency: Duration of individual GC pauses (shorter is better).**
- **Memory: Heap size and fragmentation**

JVM Parameters That Influence GC

- **-Xms256m -Xmx2048m (Heap size)**
- **-XX:+UseG1GC (GC type)**
- **-XX:MaxGCPauseMillis=100 (Pause target)**

JVM Parameters That Influence GC

■ **-XX:MaxGCPauseMillis=100 (Pause target)**

- Is a soft goal, not a hard limit, used by certain Garbage Collectors (like G1GC and ZGC) to try and keep GC pause times under a certain threshold.
- When a GC is triggered, it chooses a set of regions to collect that will maximize reclaimed space while aiming to keep the pause short.

JVM Parameters That Influence GC

■ **-XX:MaxGCPauseMillis=100 (Pause target)**

- The number and size of regions selected is adjusted dynamically to stay within the *MaxGCPauseMillis*.
- Example: If you set *-XX:MaxGCPauseMillis=50*, G1 for example might collect fewer regions during a pause to meet that *50ms* target, but this may result in more frequent collections.

Generational GCs

- **Generational GCs.**

- Young
- Old

- **Compaction**

- **Intergenerational pointers.**

Intergenerational Pointers

■ Why does it matter?

- Generational collectors often optimize by:
 - Collecting the young generation frequently (fast and cheap)
 - Collecting the old generation infrequently (slower and expensive)
- To do this efficiently, the collector only scans the old generation if necessary.
- But here's the issue: If an old object has a reference to a young object, the young object must not be collected, even though it's not referenced directly by the stack.

■ The Solution:

- These references are stored in a card table or remembered set, so during young-gen GC, the JVM can efficiently find roots in the old-gen.

Intergenerational Pointers

■ Old → Young References

- Old-gen object refers to a young-gen object
- Young generation GC does not scan the old-gen by default
- Risk: GC might reclaim young-gen object still in use
- Solution: JVM tracks these using card tables or remembered sets

■ Young → Old References

- Young-gen object refers to an old-gen object
- Old-gen is not collected during young-gen GC
- No risk of reclaiming the old object during minor GC
- These references are safe and don't need special tracking
- In the case of a major collection the GC verifies if the object is referenced from the Young-gen.

Minor and Major Collections

- **Minor Collection happens when the young generation area is near full and objects there are freed or promoted.**
- **Major Collections happen when there is need to clean the old generation area.**
 - To be able to promote objects from the young generation.

Implementations

- **Serial**
- **Parallel**
- **Concurrent Mark and Sweep**
- **G1**
- **...**

Serial GC

- **Small amount of memory required.**
- **Single thread.**
- **Generational**
- **Stop the world.**
- **Objects are compacted and allocated with bump the pointer technique.**
- **Suitable for devices with restricted resources.**
 - Does not use multiple cores during GC.

Serial GC

- **It is started automatically by the JVM in case:**

- There is less than 1792MB of RAM available
- There is only one CPU.

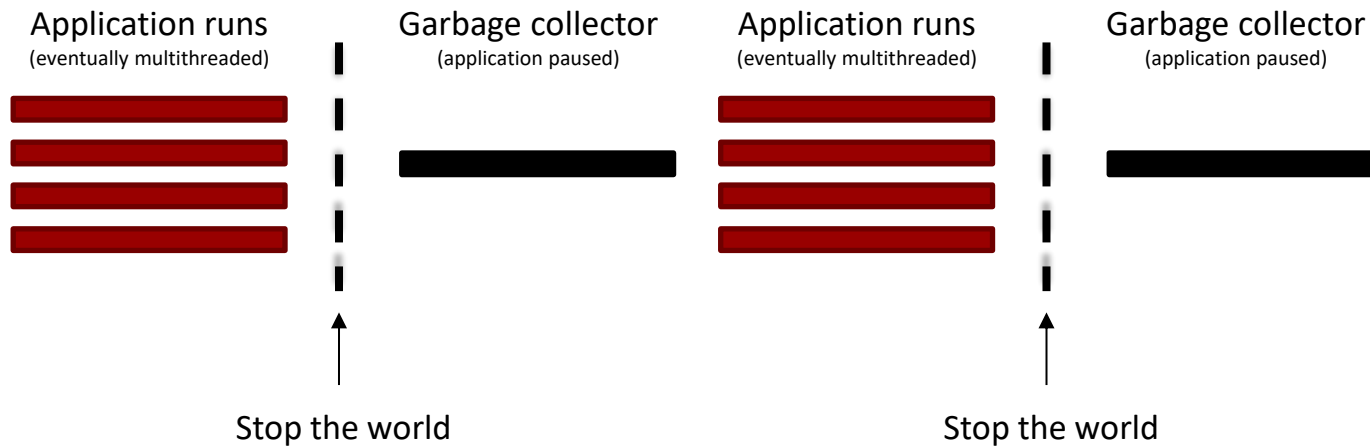
- **Otherwise:**

```
java -XX:+UseSerialGC -jar .\MyApp.java
```

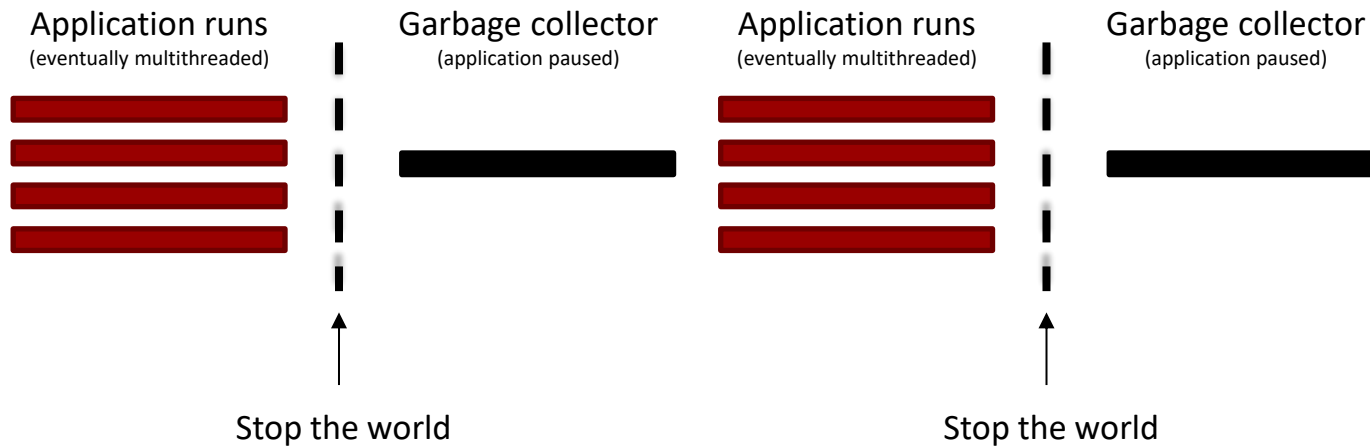
Stop the World

- For garbage collectors to operate it is necessary, for practical reasons, to periodically stop the running application so that memory can be managed.
- Depending on the algorithms, different collectors will stop-the-world at specific points of execution for varying durations of time.
- To bring an application to a total stop it is necessary to pause all the running threads.
- Garbage collectors do this by signaling the threads to stop when they come to a “safepoint”, which is a point during program execution at which all heap object contents are consistent.
- Depending on what a thread is doing it may take some time to reach a safepoint.

Stop the World



Stop the World



- This can be unacceptable for some types of applications (real-time applications, transaction processing, hardware drivers,...)

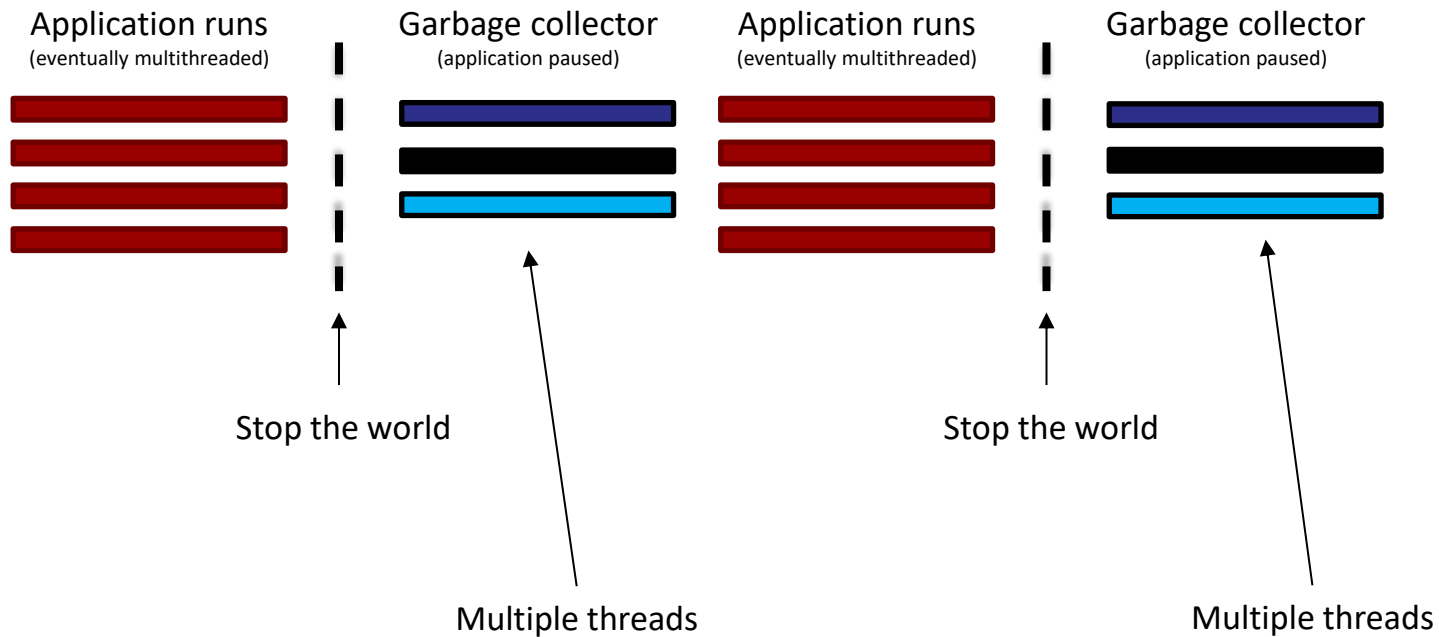
Stop the World

- Should be avoided or at least reduced.
- Parallel
- Concurrent

Parallel GC

- **Multiple threads.**
- **Does not run concurrently with the application (stop the world)**
- **Greatest throughput in multicore systems.**
- **Not suitable for applications with many interactions with the user.**
- **Good for background jobs.**

Parallel GC



Why Can It Cause Long Pauses?

- GC pauses all application threads (Stop-the-World event)
- Even if GC is fast, the app is frozen during this time
- Processing large amounts of memory = longer pause

Parallelism $\neq$ Concurrency with the App

- GC runs in parallel (multi-threaded) but not concurrently with the application
- App is paused completely while GC runs
- Faster GC still means noticeable pauses for large heaps

Impact on Responsiveness

- User interface may freeze or stutter
- Input lag and missed interactions
- Poor experience in games, multimedia, or real-time systems

Example: Music App with Parallel GC

- Minor GC every few seconds causes 200ms pause
- User hears audio glitches or notices UI lag
- Repeated pauses = frustrating experience

Parallel GC

- Multiple threads in young generation
- One thread in older generation
- Threads work in separate areas of the Heap
- Load balance to have an even distribution of work by the threads.
- When moving objects coordination is used to insure there is no conflicts.
- Some data structures (e.g. Linked Lists) must be handled sequentially and a parallel approach does not improve performance.

Parallel GC

- Can define the number of threads it uses.

```
java -XX:+UseParallelGC -XX:ParallelGCThreads=10 -jar .\MyApp.java
```

- Better to let the system choose the number of threads (based on the number of cores)

Parallel GC

- It is possible to use a previous version of the Parallel GC in order to use multiple threads both in young and old generation areas

```
java -XX:+UseParallelOldGC -jar .\MyApp.java
```

- It is possible to define the maximum amount of time spent in GC.

Concurrent Mark and Sweep

- **Runs concurrently with the application**
 - Still has “stop the world” pauses.
- **Larger memory footprint than the Parallel version**
- **Less and smaller pauses than the Parallel GC**
- **Adequate for general applications.**

How CMS Works

- **Initial Mark (STW) → marks GC roots**
 - App is paused briefly to mark GC roots.
 - Marks all objects directly reachable from the roots.
- **Concurrent Mark → runs alongside the app**
 - Marks the remaining objects
- **Remark (STW) → completes marking**
 - App is paused again to catch up on changes made during the concurrent phase.
- **Sweep → reclaims unreachable objects**

GC roots are starting points for identifying all live (reachable) objects in memory.

Advantages of CMS

- **Lower pause times than Parallel GC**
- **Application runs during most of GC cycle**
- **Good for GUIs, web servers, real-time systems**

Limitations of CMS

- Still includes STW pauses (initial mark, remark)
- No compaction → leads to memory fragmentation
- Might trigger full GC if contiguous space is unavailable

Why CMS Was Deprecated

- **Deprecated in Java 9, removed in Java 14**
- **High maintenance complexity**
 - CMS was complex and aging.
 - JVM developers had to maintain CMS in parallel with newer collectors (like G1GC, Shenandoah, ZGC).
 - It lacked modern features like region-based memory management.
- **Better alternatives available (G1GC, ZGC, Shenandoah)**
- **Suffered from fragmentation and fallback full GCs**

Concurrent Mark and Sweep

```
java -XX:+UseConcMarkSweepGC -jar .\MyApp.java
```

```
Unrecognized VM option 'UseConcMarkSweepGC'  
Error: Could not create the Java Virtual Machine.  
Error: A fatal exception has occurred. Program will exit.
```

```
java --version  
openjdk 17.0.5 2022-10-18  
OpenJDK Runtime Environment Temurin-17.0.5+8 (build 17.0.5+8)
```

Modern Alternatives to CMS

- **G1GC → Default GC since Java 9, predictable pause times**
- **Shenandoah → True concurrent compaction, low latency**
- **ZGC → Sub-10ms pauses, even for large heaps**

What is G1 GC?

- **Stands for Garbage First Garbage Collector**
- **Default collector since Java 9**
- **Focuses on balancing low pause times with high throughput**

Garbage First Collector

- Divides the heap into small regions (1MB–32MB)
- Collects regions with most garbage first
- Maximizes memory reclaimed per pause
 - and hence the name: Garbage First.

Generational Collector

- Maintains separate Young and Old generations
- Young gen: Eden + Survivor regions
- Old gen: Long-lived objects
- All part of same region pool

Evacuation (Copying)

- **G1 is a copying collector.**
- **When a young GC happens:**
 - Live objects in Eden are copied to Survivor or Old regions.
- **This copying compacts memory (eliminating fragmentation) and is called evacuation.**

Marking Collector

- **When memory pressure builds, G1 runs a concurrent marking cycle to scan old-gen regions.**

- **It marks live objects in the old generation to know:**
 - What can be reclaimed
 - Which regions are mostly garbage

Parallel Young GC

- **G1 uses multiple threads to perform young generation collections in parallel.**
- **This improves performance significantly on multi-core systems.**
- **Still a Stop-the-World (STW) event, but much shorter due to parallelism.**

Concurrent Old-Gen Marking

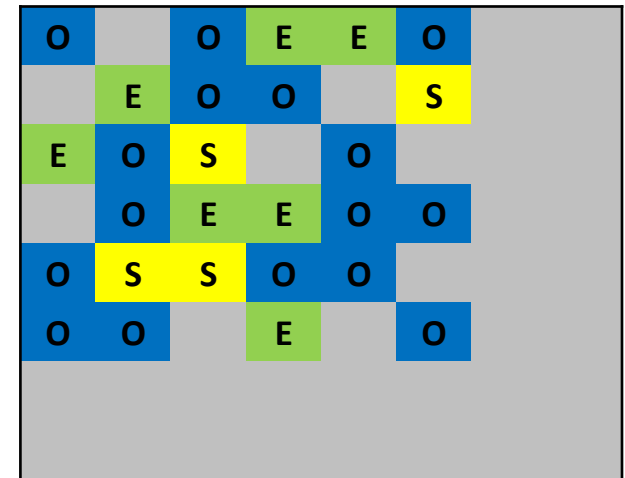
- **The old-gen marking phase is concurrent, meaning:**
 - The application keeps running while G1 is identifying live objects in the old gen.
- **This avoids long GC pauses.**
- **Only some short STW events are needed for final adjustments (like “Remark”).**

G1GC Highlights

- **Region-based, copying GC**
- **Combines parallelism + concurrency**
- **Predictable pause control with `-XX:MaxGCPauseMillis`**
- **Default collector in modern JVMs for general use**

G1 – Heap allocation

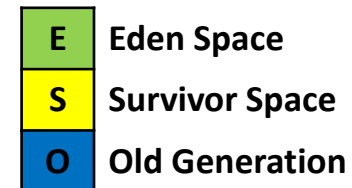
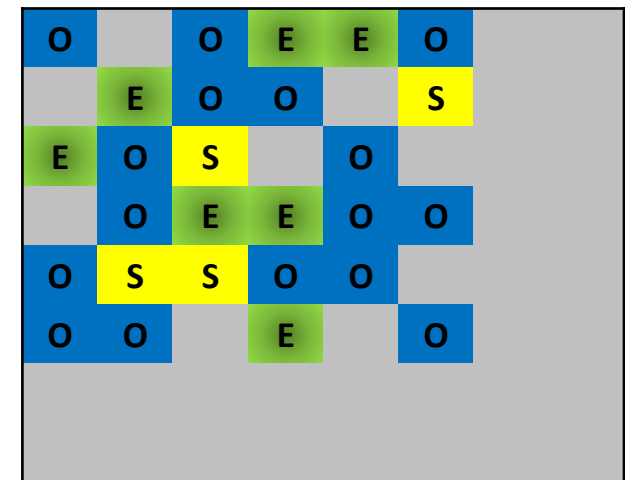
- ❑ Collection starts when all the E regions are occupied.



E	Eden Space
S	Survivor Space
O	Old Generation

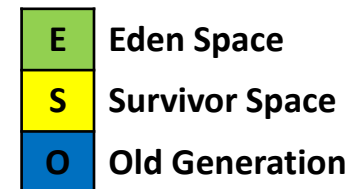
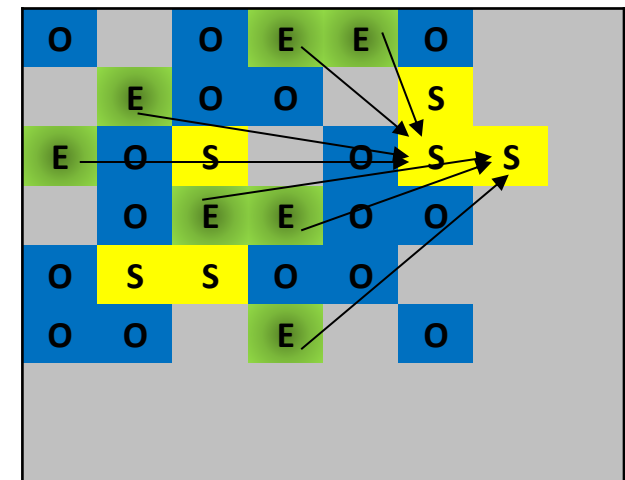
G1 – Heap allocation

- ❑ Collection starts when all the E regions are occupied.
- ❑ Evacuates E -> S and S -> O
- ❑ Young generation is Stop the World.
 - ❑ But uses multiple threads (parallel)
- ❑ Creates Survival regions to accommodate the objects alive in Eden.



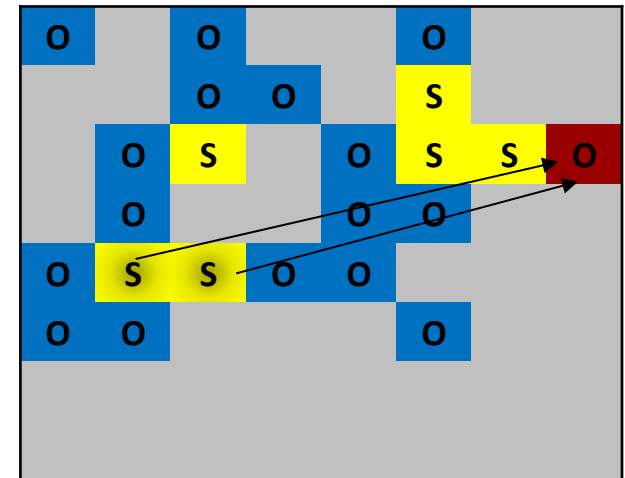
G1 – Heap allocation

- ❑ Collection starts when all the E regions are occupied.
- ❑ Evacuates E → S and S → O
- ❑ Young generation is Stop the World.
 - ❑ But uses multiple threads (parallel)
- ❑ Creates Survival regions to accommodate the objects alive in Eden.
- ❑ Only live objects are copied, let's suppose that 2 new S regions are enough.



G1 – Heap allocation

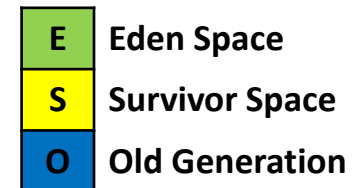
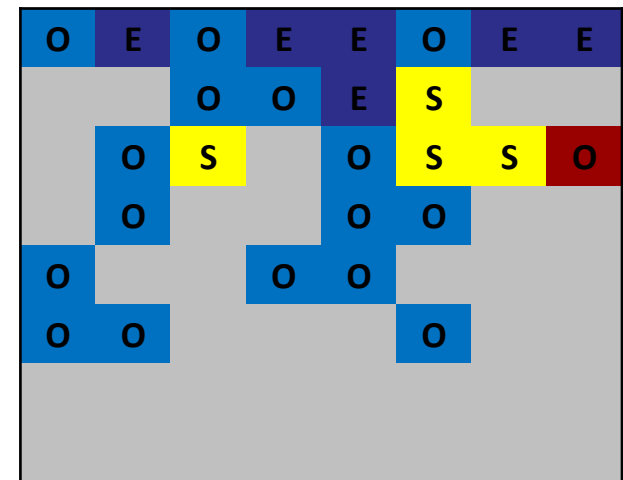
- ❑ The same approach applies to Survival regions.



E	Eden Space
S	Survivor Space
O	Old Generation

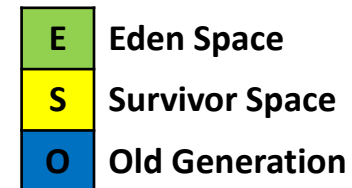
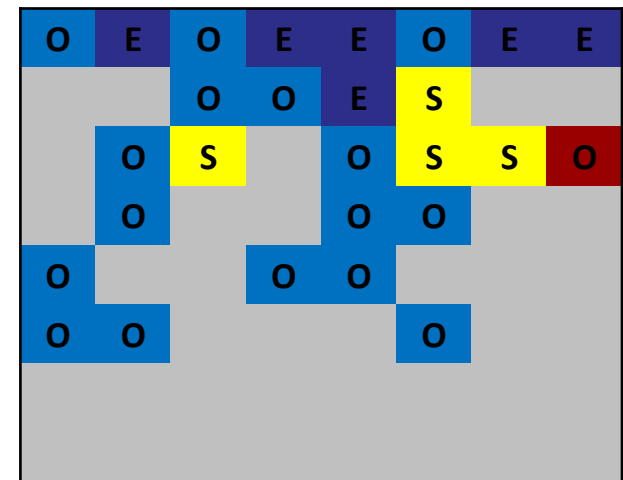
G1 – Heap allocation

- ❑ The same approach applies to Survival regions.
- ❑ New Eden generation blocks are created, and the allocation of objects proceeds.



G1 – Heap allocation

- ❑ Collection of the old generation starts when it is occupied above 45% of its total space.
- ❑ Concurrent marking while the application executes.
- ❑ Uses the same algorithm for tracing live objects that was applied in the young generation.



G1

```
java -XX:+UseG1GC -jar .\MyApp.java
```

Further implementation details can be found in:

```
https://www.oracle.com/technetwork/tutorials/tutorials-1876574.html
```

Shenandoah GC

- **Shenandoah is a low-pause, concurrent, compacting GC introduced by Red Hat, included officially in OpenJDK since Java 12.**
- **Designed for latency-sensitive applications: UIs, servers, real-time systems.**
- **Keeps pause times short regardless of heap size — suitable for large-memory workloads.**

Concurrent Relocation and Compaction

- Shenandoah can move and compact live objects while the application is still running.
- It uses concurrent evacuation, meaning it doesn't need a long STW phase for compaction.
- This makes it fundamentally different from CMS (no compaction) and G1 (STW compaction).

No Generational Division

- The heap is divided into many fixed-size regions.
- Each region can contain any kind of data — no hard boundaries like "young" or "old".
- Flexible region usage allows Shenandoah to adapt based on allocation patterns and memory pressure. Eliminates:
 - The need to promote objects
 - The need to track intergenerational pointers.

Barriers and Concurrent Operation

- Uses read and write barriers to track changes to object references during GC.
- A write barrier intercepts field updates and logs them to ensure consistency.
- A read barrier can remap references on the fly to ensure that the app always sees the correct object location.
- Trade-off:
 - These barriers add a small cost to every memory access — but enable concurrent relocation.

Configurable Collection Triggers

- Shenandoah offers tuning options to control when a GC cycle starts.
- For example:
 - Only collect regions with $\geq 60\%$ garbage
 - Start GC cycle when 80% of regions are allocated
- Helps balance performance and memory pressure.

```
java -XX:ShenandoahGarbageThreshold=60 -jar .\MyApp.java
```

```
java -XX:+ShenandoahAllocationThreshold=80 -jar .\MyApp.java
```

Shenandoah GC Highlights

- Concurrent compaction avoids long STW pauses.
- Region-based but not generational — simplifies memory management.
- Barrier-based design enables concurrent GC without data races.
- Works best in low-latency, high-heap environments (e.g. big data apps, real-time APIs).
- Optional tuning allows further control over when and how GC runs.

```
java -XX:+UseShenandoahGC -jar .\MyApp.java
```

ZGC

- Scalable, low-latency GC introduced in JDK 11+
- Pause times <10ms, regardless of heap size
- Ideal for APIs, real-time systems, cloud apps

Region-Based, Non-Generational

- Heap split into fixed-size regions
- No young/old generation split
- Simplifies GC logic and tuning

GC Phases in ZGC

- **Mark: identify live objects (concurrent)**
- **Relocate: move live objects (concurrent)**
- **Remap: fix references using barriers (concurrent)**
- **Only STW phase: root scanning (few ms)**

Pause Time Behavior

- Sub-10ms pauses common in production
- Does not grow with heap size
- Maintains responsiveness under memory pressure

Scales with Heap Size

- Efficient for multi-terabyte heaps
- Ideal for cloud-native, big-memory Java workloads

ZGC Summary

- Fully concurrent, compacting, non-generational GC
- Fast + safe relocation by using barriers
- Ultra-low pause times with minimal tuning
- Great fit for large, latency-sensitive applications

```
java -XX:+UseZGC -jar .\MyApp.java
```

GC Comparison: Serial, Parallel, G1, Shenandoah, ZGC

Feature	SerialGC	ParallelGC	G1	Shenandoah	ZGC
Concurrent			 (old-gen)		
Pause Time	High	Medium	Low	Very Low	Ultra Low
Compacting				 (concurrent)	 (concurrent)
Generational					

Further reference

<https://docs.oracle.com/en/java/javase/20/gctuning/introduction-garbage-collection-tuning.html>

The screenshot shows the Oracle Help Center interface. At the top, there's a search bar with the text "HotSpot Virtual Machine Garbage Collection Tuning Guide". Below the search bar, the breadcrumb "Java / Java SE / 20" is visible. The main heading is "HotSpot Virtual Machine Garbage Collection Tuning Guide". On the left, a "Table of Contents" sidebar lists various sections, with "1 Introduction to Garbage Collection Tuning" highlighted. The main content area displays the title "1 Introduction to Garbage Collection Tuning" with navigation arrows. Below the title, a paragraph explains that a wide variety of applications use the Java Platform, Standard Edition (Java SE), and that the Java HotSpot VM provides multiple garbage collectors. It mentions that Java SE selects the most appropriate garbage collector based on the class of the computer. A second paragraph states that general features of a garbage collector and basic tuning options are described in the context of the serial, stop-the-world collector. Below this, a "Topics" section lists three links: "What Is a Garbage Collector?", "Why Does the Choice of Garbage Collector Matter?", and "Supported Operating Systems in Documentation". The "What Is a Garbage Collector?" section is expanded, showing a paragraph about the garbage collector (GC) managing dynamic memory allocation requests, followed by a list of four operations: allocating and giving back memory, handing out memory as requested, determining which parts of memory are still in use, and reclaiming unused memory for reuse. The section concludes by stating that the Java HotSpot garbage collectors employ various techniques to improve the efficiency of these operations.

Further reference

- **"A Real-Time Garbage Collector Based on the Lifetimes of Objects" by Henry Lieberman and Carl Hewitt (1983)**
- **"Garbage Collection in an Uncooperative Environment" by Hans-Juergen Boehm and Mark Weiser (1988)**
- **"The Garbage-First Garbage Collector" by David Detlefs et al. (2004)**
- **"Shenandoah: An open-source concurrent compacting garbage collector for OpenJDK" by Christine H. Flood et al. (2016)**
- **"Deep Dive into ZGC: A Modern Garbage Collector in OpenJDK" by Albert Mingkun Yang and Tobias Wrigstad (2022)**